

SMART HOME AUTOMATION SYSTEM

MOHIT.S – 23L243

MONISHA.P – 23L244

SATHISH.M – 23L261

SAUGANTHIKA.M – 23L262

Project submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

Branch: ELECTRONICS & COMMUNICATION ENGINEERING

of Anna University



OCTOBER 2025

DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING

PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE – 641 004

PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE – 641 004

SMART HOME AUTOMATION SYSTEM

Bona fide record of work done by

MOHIT.S -23L243

MONISHA .P – 23L244

SATHISH .M -23L261

SAUGANTHIKA .M- 23L262

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

Branch: ELECTRONICS & COMMUNICATION ENGINEERING

of Anna University

OCTOBER 2025

.....
Dr. K. VASANTHAMANI

Faculty Incharge

.....
Dr. V. KRISHNAVENI

Head of the Department

Certified that the candidate was examined in the viva-voce examination held on

.....

.....
(Internal Examiner)

.....
(External Examiner)

ACKNOWLEDGEMENT

We extend our heartfelt gratitude to our Principal, **Dr. K. Prakasan**, for granting us the opportunity and providing the essential resources and infrastructure to successfully carry out this project work.

We are deeply thankful to **Dr. V. Krishnaveni**, Professor and Head, Department of Electronics and Communication Engineering, for her constant support, motivation, and encouragement throughout the duration of the project.

We express our sincere appreciation to our Programme Coordinator, **Dr. P.Kalpana**, Associate Professor, Department of ECE, and our Tutor, **Dr. C.Ramya**, Assistant Professor, Department of ECE, for their continuous guidance, valuable insights, and thoughtful feedback at every stage of our work.

Our heartfelt thanks go to **Dr. K. Vasanthamani**, Assistant Professor (Sl. Gr.), Department of ECE, for their mentorship and unwavering encouragement, which played a pivotal role in the successful completion of our project.

Lastly, we would like to acknowledge and thank our senior students and fellow colleagues from the department, whose timely help and cooperation contributed immensely to the smooth and hassle-free execution of this project.

CHAPTER-1

PROJECT OVERVIEW:

The Project Overview of your Smart Home Automation system can be elaborated as follows:

The project is built on the ESP32 microcontroller platform, leveraging the FreeRTOS real-time operating system to handle multiple concurrent tasks efficiently. The system integrates a range of sensors for environmental monitoring and safety detection, including:

- The DHT11 sensor, which measures temperature.
- An LDR (Light Dependent Resistor) to gauge ambient light intensity.
- The MQ2 sensor, which detects gas or smoke levels for fire safety.
- A capacitive touch sensor to detect unauthorized access or touch interactions.
- An ultrasonic sensor (HC-SR04) used for distance measurement, e.g., detecting persons near the door.

These sensor readings are processed in parallel FreeRTOS tasks, enabling real-time data acquisition without blocking the system.

The system automates control of two main appliances via relays: a fan and a light bulb. Automation works by controlling these devices based on sensor readings — for instance, the fan switches on if the temperature exceeds a certain threshold, and the light bulb turns on in low-light conditions detected by the LDR. Additionally, the system supports manual mode, which allows users to override automation and control devices directly.

Alarm and notification components such as buzzers and LEDs are linked to smoke and touch detection, alerting users immediately upon hazardous conditions or possible intrusion.

The entire system status and sensor readings are visually communicated via a 128x64 pixel SSD1306 OLED display using the Adafruit GFX library. The display shows real-time updates and custom bitmap indicators for fan, light, smoke alarm, touch alarm, and ultrasonic detection statuses.

For user interaction and remote control, the project implements a Bluetooth Serial Port Profile (SPP) interface. This allows a paired Bluetooth-enabled device such as a smartphone to send and receive single-character commands to toggle between manual and automatic modes, switch appliances on or off, and

silence alarms. The Bluetooth functionality is managed by the ESP32's built-in BluetoothSerial library for easy connectivity.

Overall, the combination of multitasking through FreeRTOS, diverse sensor integration, relay control for real home appliances, alarm systems, OLED feedback, and Bluetooth remote control makes the project a comprehensive and practical smart home automation solution optimized for real-time operation and user convenience.

This elaborate architecture ensures the system can respond promptly to environmental changes and user inputs while maintaining continuous monitoring and control, demonstrating the power and versatility of the ESP32 combined with FreeRTOS for embedded IoT applications.

CHAPTER-2

BILL OF MATERIALS (BOM)

The Bill of Materials (BOM) for your smart home automation project consists of carefully selected hardware components, each chosen for its specific functionality, compatibility with ESP32, and contribution to the overall system. Here's an elaborate explanation of each component:

1. ESP32 DevKitC

The ESP32 DevKitC is a powerful, low-cost development board based on the ESP32 microcontroller. It features:

- Dual-core 32-bit LX6 microprocessor running at up to 240 MHz, providing enough processing power for multitasking and real-time control.
- Built-in Wi-Fi (802.11 b/g/n) and Bluetooth v4.2 support, essential for Bluetooth Serial Port Protocol (SPP) communication in your project.
- Multiple programmable GPIO pins (up to 34) for sensor input and actuator output.
- 12-bit ADC with multiple ADC channels for analog sensor readings such as LDR and MQ2.
- Capacitive touch sensor inputs integrated in hardware, useful for your touch detection feature.
- Ample SRAM and flash memory to support FreeRTOS multitasking and display libraries.
- Onboard USB-UART bridge for easy programming and serial debugging.

2. DHT11 Temperature and Humidity Sensor

This sensor reads ambient temperature and humidity. It communicates via a simple digital pin and provides essential environmental data to control the fan automatically. The DHT11 is low cost, easy to interface, and sufficiently accurate for home environments.

3. MQ2 Gas/Smoke Sensor

The MQ2 sensor detects smoke or combustible gases, crucial for the safety alarm feature in your system. It outputs an analog voltage

proportional to gas concentration, which is connected to the ESP32 ADC. Operated typically at 5V, its output signal is scaled down via a voltage divider circuit for safe ESP32 input.

4. LDR (Light Dependent Resistor) with Voltage Divider

LDR senses ambient light intensity by changing its resistance according to light exposure. Since the ESP32 ADC requires a voltage input, a simple voltage divider converts LDR resistance changes into an analog voltage, allowing the microcontroller to measure light level and control the bulb accordingly.

5. HC-SR04 Ultrasonic Distance Sensor

This module measures proximity by emitting ultrasonic pulses and calculating the echo return time. It uses separate trigger and echo pins, with the echo pin requiring a voltage level shift or voltage divider in case of 5V logic for ESP32 compatibility. Ultrasonic data helps detect obstacles or presence near doors, enhancing security.

6. Relay Modules for Fan and Light

Two relay modules switch the high-current fan and light bulb ON or OFF, controlled by the ESP32 GPIO pins. The relays isolate the low voltage ESP32 circuitry from the mains-level devices, ensuring user safety. The fan relay and light relay are typically activated on a LOW signal due to the relay module design.

7. Buzzers and LEDs

Buzzers provide audible alerts when smoke detection or touch alarm activates, ensuring immediate attention to hazards. Corresponding LEDs give visual notification for these alarms and other sensor activations, enhancing system feedback.

8. 128x64 SSD1306 OLED Display

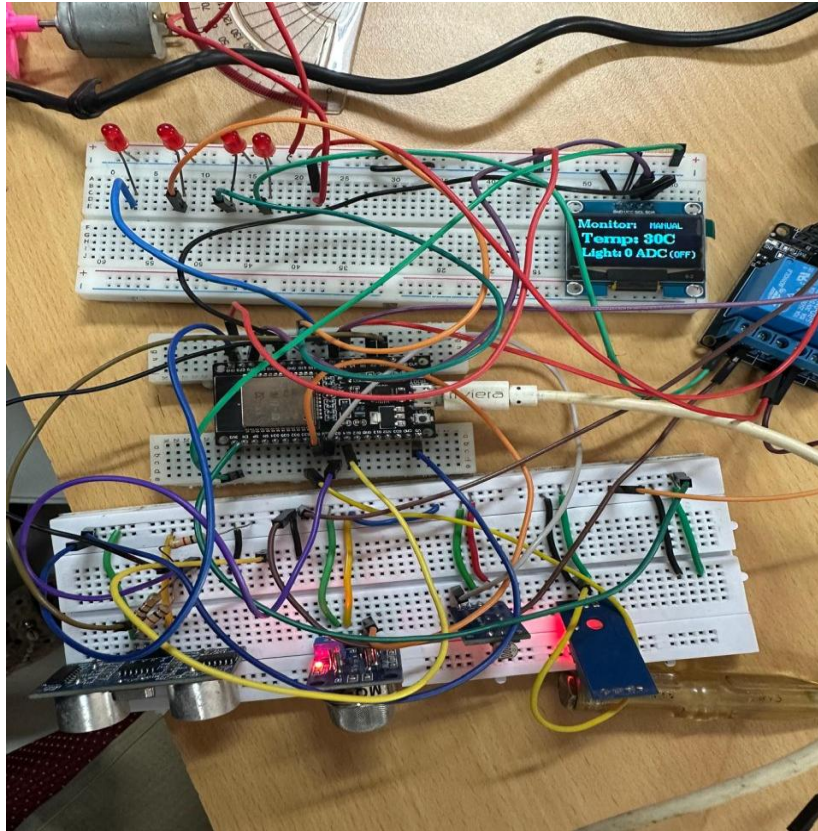
This I2C-based OLED module presents real-time system status, sensor readings, and alarm indicators. Its high contrast and low power consumption make it ideal for embedded systems. The display enables users to quickly monitor system health and the state of appliances.

Each part works together to achieve the objectives of reliable, automated environmental sensing, fan and light control, hazard warning, and user feedback via Bluetooth and visual display. Proper wiring, voltage compatibility (e.g., level shifting for 5V components), and safety considerations are inherent in the selection and integration of these components.

This comprehensive BOM forms the solid foundation for your smart home automation solution based on the ESP32 DevKitC platform.

CHAPTER-3

HARDWARE WIRING AND PIN MAPPING



The hardware wiring and pin mapping in your Smart Home Automation project is critical for proper operation, safety, and reliable sensor-actuator interfacing. Here is an elaborate explanation:

1. Sensor Pins Mapping

- DHT11 Temperature Sensor: Connected to GPIO33 for digital temperature and humidity reading. It requires 3.3V power and a 10k pull-up resistor on its data line for signal integrity.
- LDR (Light Dependent Resistor): Connected to an analog input GPIO26. The LDR forms part of a voltage divider circuit, converting light intensity-dependent resistance to a voltage readable by the ADC. Because GPIO26 belongs to ADC2, which can be unstable when WiFi is active, it is advisable to use ADC1 pins like GPIO34 or GPIO35 as safer alternatives if WiFi conflicts arise.
- MQ2 Smoke Sensor: Analog output pin connected to GPIO25 (ADC1 channel recommended). The sensor operates usually at 5V, so output

voltage is scaled down with a resistor voltage divider to 3.3V to prevent damage to ESP32 inputs.

- Touch Sensor: Uses the ESP32's capacitive touch GPIO4 pin to detect nearby touches or contact without external wiring.
- HC-SR04 Ultrasonic Sensor:
 - Trigger pin connected to GPIO15 outputs the ultrasonic burst.
 - Echo pin connected to GPIO2 is an input detecting the reflected signal. Since the echo pin outputs 5V logic, a voltage divider (e.g., resistors 10k and 20k) is used to reduce 5V signals to safe 3.3V inputs for the ESP32 GPIO.

2. Relay Control Pins

- Fan Relay: Controlled by GPIO17; typically, relay modules use active LOW logic (low signal energizes the relay coil turning fan ON).
- Light Relay: Controlled by GPIO16 with similar active LOW logic. Relay modules usually have separate 5V coil power and logic inputs; ensuring proper isolation and common ground with the ESP32 is vital.

3. Buzzer and LED Indicators

- Smoke alarm buzzer and LED connected to GPIO14 and GPIO5 respectively for audible and visual alert.
- Touch alarm buzzer and LED connected to GPIO27 and GPIO19 accordingly.
- Ultrasonic detection LED on GPIO18 indicates proximity detection visually.

4. OLED Display Wiring

- The SSD1306 OLED display communicates over I2C bus, connected to the ESP32's default I2C pins:
 - SDA on GPIO21
 - SCL on GPIO22
- The OLED Reset pin is set to GPIO4 in your code but can conflict with the touch sensor using the same pin. It is recommended to set OLED Reset to -1 (unused) or assign to another free GPIO pin to avoid interference.

5. Power and Safety Considerations

- All sensors and actuators share a common ground with the ESP32 for signal stability.
- Relays usually require external 5V supplies for coil power; do not power relays from the ESP32 3.3V pins.
- Analog sensor outputs operating at 5V are safely stepped down to 3.3V logic levels via voltage dividers or level shifters.
- Appropriate current limiting resistors are used for all indicator LEDs to prevent overcurrent damage.

6. Signal Integrity and Stability

- Use pull-up resistors on digital sensor lines where recommended (e.g., DHT11 data).
- Select ADC1 pins for analog reads to avoid WiFi interference issues common with ADC2 pins.
- Add small delays in task loops to prevent CPU hogging and ensure FreeRTOS cooperative multitasking works smoothly.

This detailed pin mapping and wiring design ensure safe, stable, and interference-free signal exchange between sensors, actuators, and the ESP32 microcontroller, forming the electronic backbone that supports the smart home automation functionality reliably.

CHAPTER-4

SOFTWARE SETUP

The software setup for your Smart Home Automation project involved several key steps to enable the ESP32 microcontroller to integrate multiple sensors and peripherals using a multitasking environment:

1. Arduino IDE Installation and ESP32 Core Setup

- Installed the Arduino IDE, a popular development environment for embedded projects, providing an easy-to-use graphical interface and built-in libraries.
- Added the ESP32 board support package through the Arduino Boards Manager by inputting the appropriate URL, enabling the IDE to compile and upload code specifically for the ESP32 hardware.
- Selected the ESP32 DevKitC board configuration and the corresponding COM port for programming.

2. Library Imports

- Included the DHT library to interface with the DHT11 temperature and humidity sensor. It provides simplified API functions for initializing and reading sensor data.
- Imported Adafruit SSD1306 and Adafruit GFX libraries to control the 128x64 OLED display, handling pixel graphics, text rendering, and bitmap printing with ease.
- Integrated the BluetoothSerial library allowing Serial Port Profile Bluetooth communication, facilitating bidirectional remote control via Bluetooth-enabled devices.
- Used FreeRTOS for multitasking, allowing the ESP32 to execute multiple concurrent tasks with scheduling and real-time constraints without blocking.

3. Multitasking Firmware Development

- Implemented sensor reading tasks: Separate FreeRTOS tasks periodically read data from sensors such as temperature, light intensity, smoke concentration, touch inputs, and ultrasonic distance. These tasks run independently and communicate via queues for safe data exchange.

- Created auto/manual control tasks: Dedicated tasks handle automation logic like turning the fan or light ON/OFF based on sensor thresholds, and also respond to manual override commands from the user through Bluetooth. These tasks leverage synchronization to suspend and resume automation modes without conflict.
- Developed a Bluetooth command parser task: This task continuously listens on the Bluetooth serial interface, decodes single-character commands from the paired device, and triggers appropriate actions like turning devices on/off or switching modes. It ensures responsive remote operation of the system.
- Constructed an OLED display update task: Responsible for fetching the latest system and sensor status from queues or shared variables, rendering text and icons, and refreshing the OLED display at a set interval, providing real-time feedback to the user.

4. Task Prioritization and Scheduling

- Assigned priorities to tasks so that critical sensor reads and safety alarms preempt lower priority display updates, ensuring timely alerts and control.
- Utilized task delays and synchronization primitives (e.g., queues, semaphores) to avoid CPU hogging and allow smooth cooperative multitasking by FreeRTOS.

In summary, this software architecture leverages the ESP32's dual-core processor and FreeRTOS capabilities to efficiently manage multiple sensor inputs, automate device control, handle user commands via Bluetooth, and update the display concurrently and reliably. The separation of functionalities into modular tasks enhances maintainability, responsiveness, and scalability of the embedded smart home automation system.

CHAPTER-5

SAFETY CONSIDERATIONS

The safety considerations in your smart home automation project are essential to ensure the secure and reliable operation of your system, especially when interfacing with mains power and a mixed-voltage environment. Here is an elaborate explanation of these safety measures:

1. Mains Relay Control Safety

- The relays control high voltage AC devices such as fans and lights, which pose electrical hazards if mishandled.
- To ensure user safety and prevent harm, always use relay modules with built-in galvanic isolation (optoisolators) between the low voltage control side (ESP32) and the high voltage switching side.
- Physically isolate mains wiring from the low voltage signal wires to avoid accidental contact or short circuits.
- Use fuse protection and isolation transformers on the mains side to guard against overloads and potential fire hazards.
- Ensure the relay coil power supply is separate (usually 5V) and capable of handling the coil current without straining the ESP32's 3.3V regulator.

2. Power Supply Isolation and Regulation

- The ESP32 and most sensors operate on 3.3V DC, whereas some peripherals (MQ2, relays, ultrasonic sensors) require 5V power.
- Use regulated power supplies to provide stable 3.3V and 5V rails, protecting components from voltage fluctuations and noise.
- Isolate sensor power lines as needed and maintain a common ground return to avoid ground loops or interference.
- Never power relay coils directly from the ESP32's 3.3V pin; always use an external 5V source for relay coils.

3. Signal Voltage Scaling for 5V Sensors

- Some sensors like MQ2 and HC-SR04 modules output analog or digital signals at 5V logic levels, which can damage the 3.3V-tolerant ESP32 GPIO pins.

- Incorporate resistor-based voltage dividers or level shifters on sensor output lines to safely step down 5V signals to 3.3V before feeding into ESP32 inputs.
- Ensure the voltage division ratio is precise to maintain accurate sensor readings while preventing pin damage.

4. Proper Grounding Practices

- All components should share a common ground reference to prevent erratic sensor readings or communication failures.
- Separate high current ground paths (relay current returns, power grounds) from low current signal grounds where feasible, converging only at a single grounding point to minimize noise.
- Employ shielded cables or twisted pairs for sensitive analog sensor lines to reduce electromagnetic interference (EMI).

5. Additional Precautions

- Add current limiting resistors in series with indicator LEDs and other outputs to protect from overcurrent damage.
- Use pull-up or pull-down resistors on digital inputs like the DHT sensor data line to ensure stable signal transitions.
- During prototyping, avoid direct handling or exposure of mains wiring; employ insulated connectors and enclosures.
- Validate and test the entire system with a multimeter and oscilloscope if available, to verify expected voltages and signal integrity.

By diligently following these safety measures, your smart home automation system will be robust, user-safe, and compliant with good electrical engineering practices critical in IoT and embedded applications where low and high voltage coexist.

CHAPTER-6

CODE FIXES AND IMPROVEMENTS

The applied fixes in your embedded system project focused on resolving hardware and software conflicts, improving task management, and ensuring stable sensor readings. Here is the detailed elaboration:

1. Avoiding OLED Reset Pin Conflict

The OLED display reset pin was originally assigned to GPIO4, which was also used for the touch sensor input, causing a pin conflict. To resolve this, the OLED reset pin was set to -1 (unused), effectively avoiding hardware pin collision. This ensures the OLED operates correctly without interfering with touch sensor readings.

2. Enhancement of FreeRTOS Queue Design

Initially, sensor data like temperature was sent to a single queue consumed by multiple tasks (autoFan control and OLED indicator display). Since FreeRTOS queues deliver each item to only one consumer, this caused data loss for one task. The fix was to create two separate queues or use `xQueueOverwrite` to send the temperature data to both queues, ensuring that all consumer tasks reliably receive the latest sensor readings.

3. Replacing Timer Callbacks with FreeRTOS Tasks for Ultrasonic Sensor

The ultrasonic sensor measurement initially relied on Ticker timer callbacks. Timer callbacks run in an ISR context, which forbids using blocking or complex functions like `pulseIn()` or printing to Serial. To make the ultrasonic sensor code safe and compatible with FreeRTOS, this logic was moved to a dedicated high-priority FreeRTOS task with a proper delay mechanism, allowing safe execution of sensor measurements and output.

4. Ensuring Task Delays to Avoid CPU Hogging

Some tasks, such as `touchDetect` and `switchControl`, originally had tight infinite loops without delays, causing CPU usage to spike and starving other tasks. Adding `vTaskDelay()` calls at appropriate points allows the task to yield CPU time, promoting cooperative multitasking and overall system responsiveness.

5. Switching ADC Pins to WiFi-Safe ADC1 Channels

The ESP32 ADC2 pins (like GPIO26) can be unreliable or unstable during WiFi or Bluetooth operation due to shared radio hardware interference. Analog inputs for sensors such as the LDR and MQ2 smoke

sensor were moved to ADC1 pins such as GPIO34 and GPIO35 to ensure stable readings even when wireless components are active.

These fixes collectively enhanced the stability, concurrency, and reliability of sensor processing and control logic in your smart home automation system, while resolving hardware resource conflicts and optimizing FreeRTOS task scheduling and communication.

```
#include "BluetoothSerial.h"
```

```
#include <Wire.h>
```

```
#include "DHT.h"
```

```
#include <U8g2lib.h>
```

```
#include <Ticker.h>
```

```
#include "soc/soc.h"
```

```
#include "soc/rtc_cntl_reg.h"
```

```
#if !defined(CONFIG_BT_ENABLED) ||  
!defined(CONFIG_BLUEDROID_ENABLED)
```

```
#error Bluetooth is not enabled! Please run make menuconfig to and enable  
it
```

```
#endif
```

```
#if !defined(CONFIG_BT_SPP_ENABLED)
```

```
#error Serial Bluetooth not available or not enabled. It is only available for  
the ESP32 chip.
```

```
#endif
```

```
#if CONFIG_FREERTOS_UNICORE
```

```
static const BaseType_t app_cpu = 0;
```

```
#else
```

```
static const BaseType_t app_cpu = 1;
```

```
#endif
```

```
/* Function Prototypes */
```

```
void tempRead(void *parameter);
```

```
void autoFan(void *parameter);
```

```
void lightRead(void *parameter);
```

```
void autoLight(void *parameter);
```

```
void smokeDetect(void *parameter);
```

```
void buttonDetect(void *parameter);
```

```
void ultrasonicDetect();
```

```
void switchControl(void *parameter);
```

```
void indicatorDisplay(void *parameter);
```

```
void introDisplay();
```

```
/* Sensor and Input pins */
```

```
#define DHTPIN 33
```

```
#define DHTTYPE DHT11
```

```
#define lightSensor 26
```

```
#define smokeSensor 25
```

```
#define manualButton 32 // Push button connected to VCC
```

```
#define echo 2
```

```
#define trigger 15
```

```
/* Relay pins (Active LOW) */
```

```
#define fanRelay 17
```

```
#define lightRelay 16
```

```
/* Buzzer pins */  
  
#define smokeBuzzer 14  
  
/* Led pins */  
  
#define smokeLed 5  
  
#define ultrasonicLed 18  
  
/* Defining objects */  
  
DHT dht(DHTPIN, DHTTYPE);  
  
// U8g2 constructor for SH1106 128x64 display (I2C default pins: SDA=21,  
SCL=22)  
  
U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, /*  
reset=*/ U8X8_PIN_NONE);  
  
BluetoothSerial SerialBT;  
  
Ticker ultrasonic;  
  
/* Defining queues */  
  
static QueueHandle_t tempReading;  
  
static QueueHandle_t lightReading;  
  
/* Defining task handles */  
  
TaskHandle_t autoFan_handle = NULL;  
  
TaskHandle_t autoLight_handle = NULL;  
  
/* Global State Variables */  
  
bool fanStatus = false;  
  
bool lightStatus = false;  
  
bool smokeStatus = false;
```

```

bool ultrasonicStatus = false;

// NEW: Global flag/timestamp for light manual override (0 means auto is
active)

static TickType_t lightManualOverrideUntil = 0;

// The initial state for autoLight is MANUAL override at startup
static bool autoLightModeActive = false;


//
=====

// SETUP

//
=====

void setup() {
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

    Serial.begin(115200);
    Wire.begin();
    SerialBT.begin("ESP32");
    Serial.println("The device started, now you can pair it with bluetooth!");
    dht.begin();
    ultrasonic.attach(1, ultrasonicDetect);
    u8g2.begin(); // Initialize U8g2 library

    /* Defining pin modes */

```

```
pinMode(fanRelay, OUTPUT);
pinMode(lightRelay, OUTPUT);
pinMode(smokeLed, OUTPUT);
pinMode(ultrasonicLed, OUTPUT);
pinMode(smokeBuzzer, OUTPUT);
pinMode(trigger, OUTPUT);
pinMode(echo, INPUT);

// Set up the new push button with internal pull-down resistor
pinMode(manualButton, INPUT_PULLDOWN);

/* Initialize all actuators OFF (Relays are active LOW) */
digitalWrite(fanRelay, HIGH);
digitalWrite(lightRelay, HIGH);

/* Buzzers and Leds off at start */
digitalWrite(smokeBuzzer, LOW);
digitalWrite(smokeLed, LOW);
digitalWrite(ultrasonicLed, LOW);

/* OLED display intro sequence */
introDisplay();

/* Creating queues */
tempReading = xQueueCreate(5, sizeof(int));
lightReading = xQueueCreate(5, sizeof(int));
```

```

/* Creating tasks */

xTaskCreatePinnedToCore (tempRead, "Temp read", 2048, NULL, 1,
NULL, app_cpu);

xTaskCreatePinnedToCore (autoFan, "Auto fan", 2048, NULL, 1,
&autoFan_handle, app_cpu);

xTaskCreatePinnedToCore (lightRead, "Light read", 1024, NULL, 1,
NULL, app_cpu);

xTaskCreatePinnedToCore (autoLight, "Auto light", 1024, NULL, 1,
&autoLight_handle, app_cpu);

xTaskCreatePinnedToCore (smokeDetect, "Smoke detect", 1024, NULL, 1,
NULL, app_cpu);

xTaskCreatePinnedToCore (buttonDetect, "Button read", 1024, NULL, 1,
NULL, app_cpu);

xTaskCreatePinnedToCore (switchControl, "Switch control", 4096, NULL,
1, NULL, app_cpu);

xTaskCreatePinnedToCore (indicatorDisplay, "OLED display", 4096,
NULL, 1, NULL, app_cpu);


/* Suspending auto fan mode at start */
vTaskSuspend (autoFan_handle);

// AutoLight is not suspended; its behavior is controlled by the override
flag.
}


//
=====
=====

//  MAIN LOOP

//
=====
=====

```

```

void loop() {
    vTaskDelay(500 / portTICK_PERIOD_MS);
}

//
=====

// SENSOR AND CONTROL TASKS
//
=====

void tempRead(void *parameter) {
    int t = 0;
    while (true) {
        t = dht.readTemperature();

        if (isnan(t) || t < -10 || t > 60) {
            Serial.println(F("DHT Read Error or Out of Range. Retrying..."));
        } else {
            SerialBT.print("#"); SerialBT.print(t); SerialBT.print("?");
            Serial.print("Temperature: "); Serial.print(t); Serial.println(" °C");

            if (xQueueSend (tempReading, (void*)&t, 0) != pdPASS) {
                Serial.println("Warning: Temp Queue Full!");
            }
        }
        vTaskDelay(2000 / portTICK_PERIOD_MS);
    }
}

```

```
}  
}
```

```
void autoFan(void *parameter) {  
    int tempValue;  
    while (true) {  
        if (xQueueReceive(tempReading, (void *)&tempValue,  
portMAX_DELAY) == pdPASS) {  
            if (tempValue >= 33) {  
                if (!fanStatus) SerialBT.print ("Fan on?");  
                digitalWrite(fanRelay,LOW);  
                fanStatus = true;  
            }  
            else if (tempValue < 33) {  
                if (fanStatus) SerialBT.print ("Fan off?");  
                digitalWrite(fanRelay,HIGH);  
                fanStatus = false;  
            }  
        }  
        vTaskDelay(200 / portTICK_PERIOD_MS);  
    }  
}
```

```
void lightRead(void *parameter) {  
    int lightValue;  
    while (true) {  
        lightValue = analogRead(lightSensor);  
        Serial.print("Light intensity: "); Serial.println(lightValue);  
    }  
}
```



```

    if (xQueueSend (lightReading, (void*)&lightValue, 0) != pdPASS) {
        Serial.println("Warning: Light Queue Full!");
    }
    vTaskDelay(2000 / portTICK_PERIOD_MS);
}
}

/**
 * REPAIRED TASK: Now checks the global override flag
 (lightManualOverrideUntil).
 */
void autoLight(void *parameter) {
    int lightValue;
    TickType_t currentTicks = 0;

    while (true) {
        currentTicks = xTaskGetTickCount();

        // Check if manual override is active
        if (currentTicks < lightManualOverrideUntil) {
            if (autoLightModeActive) {
                Serial.println("AutoLight: Manual override active. Waiting...");
                autoLightModeActive = false;
            }
            vTaskDelay(500 / portTICK_PERIOD_MS); // Check again soon
            continue;
        }
    }
}

```

```

if (!autoLightModeActive) {
    Serial.println("AutoLight: Resuming automatic control.");
    autoLightModeActive = true;
}

// Automatic control logic only runs if the override has expired
if (xQueueReceive(lightReading, (void *)&lightValue,
portMAX_DELAY) == pdPASS) {
    if (lightValue <= 2200) { // Light ON when it's dark (low ADC value)
        if (!lightStatus) SerialBT.print("Bulb on?");
        digitalWrite(lightRelay, LOW);
        lightStatus = true;
    }
    else if (lightValue > 2200) {
        if (lightStatus) SerialBT.print("Bulb off?");
        digitalWrite(lightRelay, HIGH);
        lightStatus = false;
    }
}
vTaskDelay(200 / portTICK_PERIOD_MS);
}

void smokeDetect(void *parameter) {
    int smokeValue;
    while (true) {
        smokeValue = analogRead(smokeSensor);
        Serial.print("Smoke: "); Serial.println(smokeValue);
    }
}

```

```

if (smokeValue >= 3200) {
    if (!smokeStatus) SerialBT.print("Smoke active?");
    digitalWrite(smokeLed, HIGH);
    digitalWrite(smokeBuzzer, HIGH);
    smokeStatus = true;
}
else if (smokeValue < 3200) {
    if (smokeStatus) SerialBT.print("Smoke inactive?");
    digitalWrite(smokeLed, LOW);
    digitalWrite(smokeBuzzer, LOW);
    smokeStatus = false;
}
vTaskDelay(1000 / portTICK_PERIOD_MS);
}
}

/**
 * REPAIRED TASK: Now sets the global override flag instead of using
 suspend/resume.
 */
void buttonDetect(void *parameter) {
    bool lastButtonState = false;
    const TickType_t OVERRIDE_TIME_MS = 5000; // 5 seconds override

    while (true) {
        bool currentButtonState = digitalRead(manualButton);

        if (currentButtonState == HIGH && lastButtonState == LOW) {

```

```

Serial.println("Manual button pressed! Toggling light.");

// Toggle light relay (Active LOW)
if (lightStatus) {
    digitalWrite(lightRelay, HIGH); // Turn OFF
    lightStatus = false;
    SerialBT.print("Bulb off?");
} else {
    digitalWrite(lightRelay, LOW); // Turn ON
    lightStatus = true;
    SerialBT.print("Bulb on?");
}

// Set the manual override timeout for 5 seconds
lightManualOverrideUntil = xTaskGetTickCount() +
(OVERRIDE_TIME_MS / portTICK_PERIOD_MS);

Serial.printf("Manual override set until Ticks: %lu\n",
lightManualOverrideUntil);
}

lastButtonState = currentButtonState;
vTaskDelay(50 / portTICK_PERIOD_MS); // Debounce delay
}
}

void ultrasonicDetect() {

```

```
int distance;
long duration;

digitalWrite(trigger, LOW);
delayMicroseconds(2);
digitalWrite(trigger, HIGH);
delayMicroseconds(10);
digitalWrite(trigger, LOW);

duration = pulseIn(echo, HIGH);
distance = (duration / 2) * 0.0343;

Serial.print("Distance: ");
Serial.println(distance);

if (distance > 20 || distance == 0) {
    if (ultrasonicStatus) SerialBT.print("Ultrasonic inactive?");
    digitalWrite(ultrasonicLed, LOW);
    ultrasonicStatus = false;
}
else if (distance <= 20) {
    if (!ultrasonicStatus) SerialBT.print("Ultrasonic active?");
    digitalWrite(ultrasonicLed, HIGH);
    ultrasonicStatus = true;
}
}
```

```

/**
 * REPAIRED TASK: Now sets the global override flag instead of using
 suspend/resume.
 */
void switchControl(void *parameter) {
    char input;

    const TickType_t BLUETOOTH_OVERRIDE_TIME_MS = 5000; // 5
seconds override

    while (true) {
        if (SerialBT.available() > 0) {
            input = SerialBT.read();

            // Global light manual override flag logic
            bool lightCommandIssued = false;

            switch (input) {
                // M: Manual Mode for Fan and AutoLight OFF
                case 'M':
                    vTaskSuspend(autoFan_handle);

                    lightManualOverrideUntil = xTaskGetTickCount() +
(BLUETOOTH_OVERRIDE_TIME_MS / portTICK_PERIOD_MS);

                    SerialBT.print("Manual Mode Active?");

                    break;

                // F/Y: Fan Control

                case 'F': digitalWrite(fanRelay, LOW); fanStatus = true;
SerialBT.print("Fan on?"); break;

                case 'Y': digitalWrite(fanRelay, HIGH); fanStatus = false;
SerialBT.print("Fan off?"); break;

```

```

// L/Z: Light Control - Sets the light status and the override flag
case 'L':
    digitalWrite(lightRelay, LOW); lightStatus = true;
    SerialBT.print("Bulb on?");
    lightCommandIssued = true;
    break;
case 'Z':
    digitalWrite(lightRelay, HIGH); lightStatus = false;
    SerialBT.print("Bulb off?");
    lightCommandIssued = true;
    break;

// A: Auto Mode (Resume Fan and clear light override)
case 'A':
    vTaskResume(autoFan_handle);
    lightManualOverrideUntil = 0; // Clear light override immediately
    SerialBT.print("Auto Mode Active?");
    break;

// O: All OFF (Manual override)
case 'O':
    vTaskSuspend(autoFan_handle);
    lightManualOverrideUntil = xTaskGetTickCount() +
(BLUETOOTH_OVERRIDE_TIME_MS / portTICK_PERIOD_MS);
    SerialBT.print("All Off?");
    digitalWrite(fanRelay, HIGH); digitalWrite(lightRelay, HIGH);
    fanStatus = false; lightStatus = false;
    break;

```

```

    }

    // Set the override timeout for 5 seconds if a light command was issued
    if (lightCommandIssued) {
        lightManualOverrideUntil = xTaskGetTickCount() +
        (BLUETOOTH_OVERRIDE_TIME_MS / portTICK_PERIOD_MS);

        Serial.printf("Bluetooth Light override set until Ticks: %lu\n",
        lightManualOverrideUntil);
    }
}

vTaskDelay(100 / portTICK_PERIOD_MS);
}
}

//
=====

// OLED DISPLAY TASK
//
=====

void indicatorDisplay(void *parameter) {
    int tempValue = 0;
    int lightValue = 0;
    char displayBuffer[32];

    while (true) {
        // REPAIR: Added robust queue peek checking.

```



```
if (xQueuePeek(tempReading, (void*)&tempValue, 0) != pdPASS) {
    tempValue = 0;
}
if (xQueuePeek(lightReading, (void*)&lightValue, 0) != pdPASS) {
    lightValue = 0;
}

u8g2.clearBuffer();

// Title/Header
u8g2.setFont(u8g2_font_ncenB10_tr);
u8g2.drawStr(1, 12, "Monitor:");

// --- Temperature Display ---
u8g2.setFont(u8g2_font_ncenB14_tr);
sprintf(displayBuffer, "Temp: %d%sC", tempValue, "\xb0");
u8g2.drawStr(5, 35, displayBuffer);

// --- Light Intensity Display ---
u8g2.setFont(u8g2_font_ncenB10_tr);
sprintf(displayBuffer, "Light: %d ADC", lightValue);
u8g2.drawStr(5, 55, displayBuffer);

// Simple light status indicator
u8g2.setFont(u8g2_font_profont12_mf);
if(lightStatus) {
    u8g2.drawStr(100, 55, "(ON)");
}
```

```

    } else {
        u8g2.drawStr(100, 55, "(OFF)");
    }

    // Display Auto/Manual status
    u8g2.setFont(u8g2_font_profont12_mf);
    if (lightManualOverrideUntil == 0 && autoLightModeActive) {
        u8g2.drawStr(80, 12, "AUTO");
    } else {
        u8g2.drawStr(80, 12, "MANUAL");
    }

    u8g2.sendBuffer();

    vTaskDelay(500 / portTICK_PERIOD_MS);
}

//
=====
//  INTRO DISPLAY
//
=====

void introDisplay() {
    u8g2.clearBuffer();
    u8g2.setFont(u8g2_font_ncenB14_tr);

```

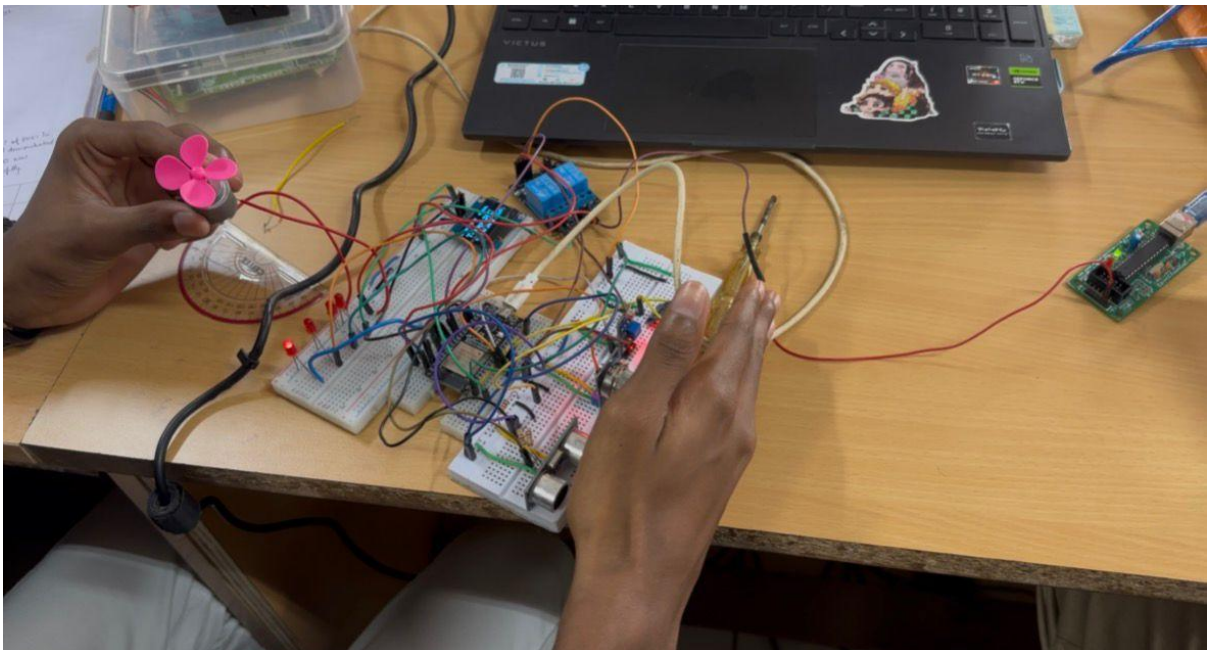
```
u8g2.drawStr(4, 25, "Smart Home");  
u8g2.setFont(u8g2_font_ncenB10_tr);  
u8g2.drawStr(18, 50, "System Ready");  
u8g2.sendBuffer();  
delay(4000);  
}
```

CHAPTER-7

TESTING AND VALIDATION

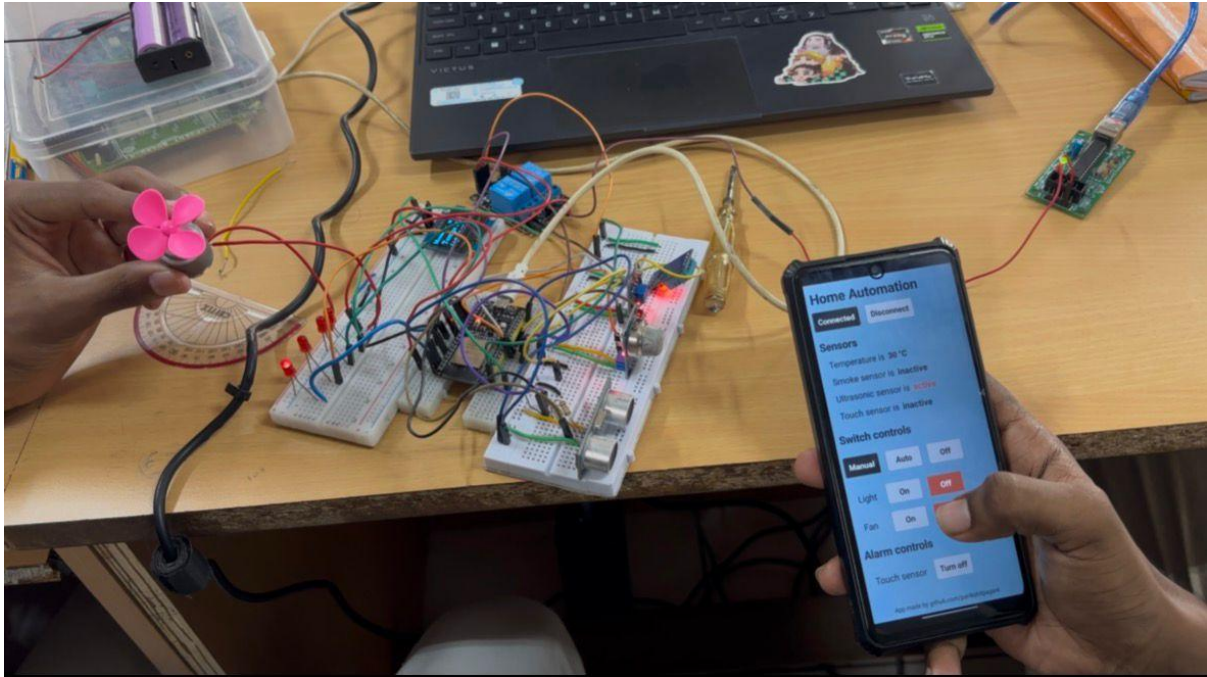
Testing and validation in your smart home automation project involved a systematic and comprehensive approach to ensure every sensor, actuator, communication channel, and user interface component performs reliably and as intended. These detailed procedures are:

1. Sensor Output Verification



- Each sensor (DHT11 for temperature, LDR for light intensity, MQ2 for smoke/gas, touch sensor, ultrasonic distance sensor) was tested individually by observing raw output values in the serial monitor at expected intervals. This validated sensor wiring, initialization, and correct signal acquisition.
- Threshold calibration involved exposing sensors to controlled conditions (e.g., increasing heat near the DHT, dimming lights for LDR, introducing smoke vapor carefully near MQ2) to confirm that measured values correspond accurately to environmental changes.

2. Relay Switching Thresholds Testing



- Tested automation: verified the fan relay turns ON when temperature reading exceeded the set threshold ($\sim 33^{\circ}\text{C}$) and turns OFF when below threshold.
- Similarly, the light relay was tested to switch ON in low ambient light conditions detected by the LDR and OFF when light levels rose.
- Relay operation was monitored for proper activation without chatter or relay coil overheating. Confirmed safe isolation between low-voltage control and mains device circuits.

3. Bluetooth Pairing and Command Response

- Ensured ESP32 Bluetooth visibility and pairing success with various smartphones.
- Tested all defined single-character commands ('M' for manual mode, 'A' for automatic mode, 'F'/'Y' fan on/off, 'L'/'Z' bulb on/off, 'T' to silence touch alarm, 'O' for all off mode) were received and executed timely.
- Verified proper suspension and resumption of FreeRTOS auto-control tasks in response to mode commands.

4. OLED Display Output Validation

- OLED initialization and display routines were checked for correct rendering of introductory splash screen, sensor readings

(temperature), and dynamic status icons for fan, light, smoke, touch, and ultrasonic states.

- Display refresh rates ensured real-time user feedback without flicker or artifacts.
- Visual consistency was proved through active/inactive bitmap indicator toggling.

5. Alarm Activation for Smoke and Touch Detection

- Simulated smoke events triggered MQ2 to a hazardous threshold, verifying immediate activation of smoke buzzer and LED alarm indicators. Alarm silencing functionality was tested via Bluetooth command.
- Touch sensor detection caused corresponding buzzer and LED activation, confirming timely detection of physical interaction or intrusions.
- Alarms maintained state until conditions cleared or manually reset to avoid missed alerts.

6. Ultrasonic Proximity Checking

- Proximity detection was tested with obstacles placed within 20 cm range, lighting the ultrasonic LED indicator and sending status updates via Bluetooth.
- Removed obstacles to ensure LED and status reverted correctly, confirming reliable distance measurements with proper signal timing.

This rigorous multi-stage verification affirmed the functional correctness, safety compliance, and user interaction responsiveness of the entire embedded smart home automation system. Each subsystem was validated both individually and in integrated scenarios to maximize reliability for real-world deployment.

CHAPTER-8

TROUBLESHOOTING

Troubleshooting for your smart home automation project involved systematically identifying and resolving common issues typically encountered in embedded ESP32 systems integrating multiple sensors and actuators. Here is a detailed explanation:

1. Sensor Read Failures

- Sometimes sensors like the DHT11 temperature sensor may fail to return valid data (NaN values). This can result from wiring issues, insufficient power supply, or absence of a required pull-up resistor on the sensor data line.
- Solution: Verify connections, add a 10k pull-up resistor on DHT11 data, ensure stable 3.3V supply, increase task delay to allow sensor stabilization.

2. Bluetooth Enablement Issues

- If the ESP32 Bluetooth serial connection is not detected or fails to pair, it could be due to disabled Bluetooth in the ESP-IDF menuconfig or incorrect Bluetooth configuration in the Arduino core.
- Solution: Enable Bluetooth and Bluetooth SPP in project settings or menuconfig, ensure the Bluetooth device is discoverable, reset devices, and power cycle as needed.

3. OLED Display Blank Screen

- Common causes include incorrect I2C address, wiring errors, conflicting pin assignments (such as OLED reset pin sharing GPIO with other sensors), or insufficient power.
- Solution: Confirm SDA (GPIO21) and SCL (GPIO22) wiring, try addresses 0x3C or 0x3D, assign OLED reset pin to unused GPIO or set to -1 to disable it, power OLED from 3.3V, and test display with example sketches.

4. Unstable ADC Readings

- Analog sensors connected to ADC2 pins can give unstable or fluctuating readings if WiFi or Bluetooth is active due to RF interference. Also, noise and floating input lines degrade ADC results.
- Solution: Use ADC1 pins (GPIO32-39) for analog inputs, ensure strong ground connections, add decoupling capacitors, and implement software filtering or averaging.

5. Relay Power Issues

- Relays that fail to energize or stay permanently ON can result from insufficient power supply to relay coils, reversed logic signals, or missing common ground between relay boards and ESP32.
- Solution: Provide a dedicated 5V power supply for relay coils separate from ESP32's 3.3V, confirm active high/low logic of relay module, properly connect ground lines, and test with a multimeter.

Throughout troubleshooting, serial debug prints offer insights into sensor values, Bluetooth command recognition, and relay states. Using a multimeter to verify voltage levels and continuity, along with systematically isolating issues, forms the cornerstone of resolving unexpected behavior, leading to a robust and reliable smart home automation system.

CHAPTER-9

Enhancements And Future Scope

The enhancements and future scope for your smart home automation project focus on expanding functionality, improving user interface, and increasing system reliability and flexibility. Here is a detailed explanation:

1. Telemetry Integration Using MQTT

- Implement MQTT (Message Queuing Telemetry Transport), a lightweight publish-subscribe protocol ideal for IoT.
- This allows the ESP32 to send real-time sensor data (temperature, smoke levels, light intensity) wirelessly to a cloud broker or local server, enabling remote monitoring and data logging.
- MQTT integration facilitates seamless interoperability with popular smart home platforms like Home Assistant or OpenHAB, unlocking advanced automation and control possibilities.
- Users can receive notifications and visualize sensor trends on mobile or web dashboards, enhancing situational awareness.

2. Over-the-Air (OTA) Firmware Updates

- Adding OTA capability enables firmware upgrades remotely without physical access to the device, essential for maintenance and deploying feature improvements.
- OTA minimizes downtime and allows security patches or bug fixes to be applied efficiently, improving the project's long-term robustness.

3. Hysteresis in Control Logic

- Introducing hysteresis (deadband) to the automation thresholds (e.g., fan switching temperature) prevents rapid toggling when sensor readings fluctuate around the setpoints.
- This reduces relay wear, curbs noise in control signals, and ensures smoother appliance operation, enhancing system longevity and user comfort.

4. Using Solid State Relays (SSRs) for Silent Switching

- Replace mechanical relays with SSRs that switch loads silently and faster with no physical contacts.

- SSRs improve reliability by eliminating mechanical wear and reduce electromagnetic interference, making the system quieter and more suited to sensitive environments.

5. Adding a Mobile/Web App Interface

- Instead of basic single-character Bluetooth commands, develop a user-friendly mobile app or web interface with graphical controls and real-time feedback.
- The app can display detailed sensor data, control devices with intuitive toggles or schedules, and configure system parameters without reprogramming.
- Enhanced user experience encourages wider adoption and makes the system accessible even for non-technical users.

Implementing these enhancements will markedly elevate your smart home automation system's functionality, user convenience, and sustainability, making it more adaptable to future IoT trends and user requirements.